

CL249 2026 Date: 4/8/2026

Assignment 1-Numerical Solving of Linear Systems

There three distinct assignment. All students are provided with a broken script for LU decomposition with Pivoting. All students have to debug and correct the broken code. Each variation contains additional questions.

Marks- Debugging (2), Testing (1), Additional questions (different for variation) (2). Viva (5)

import numpy as np

```
def lu_factorization_flawed(A):
```

```
    """
```

Computes LU Decomposition of a square matrix A.

EXPECTED OUTPUT:

P: Permutation matrix (n x n)

L: Lower triangular matrix with 1s on main diagonal (n x n)

U: Upper triangular matrix (n x n)

```
    """
```

```
    n = A.shape[0]
```

```
    L = np.zeros((n, n))
```

```
    U = np.copy(A).astype(float)
```

```
    P = np.eye(n)
```

```
    for i in range(n):
```

```
        for j in range(i + 1, n):
```

```
            factor = U[j, i] / U[i, i]
```

```
            L[i, j] = factor
```

```
            # Row elimination on U
```

```
            U[j, i:] -= factor * U[i, i:]
```

```
    return P, L, U
```

```
def forward_substitution_flawed(L, b):
    """
    Solves  $Ly = b$  for  $y$  using Forward Substitution.
    """
    n = L.shape[0]
    y = np.zeros(n)
    for i in range(n):
        s = np.dot(L[i, :i], y[:i])
        y[i] = b[i] - s
    return y
```

```
def backward_substitution_flawed(U, y):
    """
    Solves  $Ux = y$  for  $x$  using Backward Substitution.
    """
    n = U.shape[0]
    x = np.zeros(n)

    for i in range(n):
        s = np.dot(U[i, i+1:], x[i+1:])
        x[i] = (y[i] - s) / U[i, i]

    return x
```

```
def solve_system_flawed(A, b):
    # Solves  $Ax = b$  using LU decomposition.
    P, L, U = lu_factorization_flawed(A)
```

```

Pb = np.dot(P, b)

y = forward_substitution_flawed(L, Pb)

x = backward_substitution_flawed(U, y)

return x

```

```

# =====

# CODE TESTING

# =====

if __name__ == "__main__":

    print("--- RUNNING FLAWED STARTER CODE TESTS ---")


    # Test 1: Standard 3x3 Matrix

    A1 = np.array([

        [2.0, 1.0, 1.0],

        [4.0, -6.0, 0.0],

        [-2.0, 7.0, 2.0]

    ])

    b1 = np.array([5.0, -2.0, 9.0])


    print("\n[Test 1] Standard 3x3 Matrix:")

    try:

        x1 = solve_system_flawed(A1, b1)

        print("Computed x:", x1)

        print("Residual ||Ax - b||:", np.linalg.norm(np.dot(A1, x1) - b1))

        # we are using linear algebra package here, But do not use it elsewhere unless told

    except Exception as e:

        print("FAILED with error:", e)

```

```

# Test 2: Matrix requiring row swapping (Zero pivot at A[0,0])
A2 = np.array([
    [0.0, 2.0, 1.0],
    [1.0, -2.0, -1.0],
    [2.0, 4.0, 1.0]
])
b2 = np.array([4.0, -1.0, 7.0])

print("\n[Test 2] Zero-Pivot Matrix (Requires Partial Pivoting):")
try:
    x2 = solve_system_flawed(A2, b2)
    print("Computed x:", x2)
    print("Residual ||Ax - b||:", np.linalg.norm(np.dot(A2, x2) - b2))
except Exception as e:
    print("FAILED with error:", e)

```

ALGORITHM ForwardEliminationPivot(A, b):

INPUT: Matrix A of size $n \times n$, Vector b of size n

OUTPUT: Upper Triangular Matrix U, Modified Vector b

U = copy(A)

FOR k = 1 TO n - 1 DO

 // -----

 // STEP 1: PARTIAL PIVOTING (Find row with max entry in column k)

 // -----

 max_row = k

 FOR i = k + 1 TO n DO

 IF $|U[i, k]| > |U[\text{max_row}, k]|$ THEN

 max_row = i

 END IF

 END FOR

```

// Check if matrix is singular
IF |U[max_row, k]| < EPSILON THEN
    RAISE ERROR "Matrix is singular or near-singular!"
END IF

// Swap pivot row with current row k (if necessary)
IF max_row != k THEN
    SWAP row U[k, :] AND row U[max_row, :]
    SWAP element b[k] AND element b[max_row]
END IF

// -----
// STEP 2: ROW ELIMINATION
// -----
FOR i = k + 1 TO n DO
    factor = U[i, k] / U[k, k]

    FOR j = k TO n DO
        U[i, j] = U[i, j] - factor * U[k, j]
    END FOR

    b[i] = b[i] - factor * b[k]
END FOR

END FOR
// Final check for singularity
IF |U[n, n]| < EPSILON THEN
    RAISE ERROR "Matrix is singular or near-singular!"
END IF

RETURN U, b

```

Next Part- Back Substitution

Start with :

$x_n = b_n / U_{n,n}$ (You must have $U_{n,n} \neq 0$)

$$x_i = \frac{b_i - \sum_{j=i+1}^n U_{i,j} x_j}{U_{i,i}}$$

BackSubstitution(U, b):

INPUT: Upper Triangular Matrix U of size n x n, Modified Vector b of size n

OUTPUT: Solution Vector x of size n

Initialize Vector x of size n

// 1. Solve for the last variable x_n

$x[n] = b[n] / U[n, n]$

// 2. Iterate backwards from row $n-1$ down to 1

FOR $i = n - 1$ DOWNTO 1 DO

$sum = 0$

 FOR $j = i + 1$ TO n DO

$sum = sum + U[i, j] * x[j]$

 END FOR

$x[i] = (b[i] - sum) / U[i, i]$

END FOR

RETURN x

LU Decomposition

DoolittleLU(A):

INPUT: Square Matrix A of size $n \times n$

OUTPUT: Lower Triangular Matrix L, Upper Triangular Matrix U

Initialize L as an $n \times n$ zero matrix

Initialize U as an $n \times n$ zero matrix

FOR $k = 1$ TO n DO

// 1. Set main diagonal of L to 1

$L[k, k] = 1$

// 2. Compute elements of U in row k ($j \geq k$)

FOR $j = k$ TO n DO

$sum = 0$

 FOR $m = 1$ TO $k - 1$ DO

$sum = sum + L[k, m] * U[m, j]$

 END FOR

$U[k, j] = A[k, j] - sum$

END FOR

// Check for zero pivot to prevent division by zero

IF $|U[k, k]| < \text{EPSILON}$ THEN

 RAISE ERROR "Zero pivot encountered! Doolittle LU decomposition fails without pivoting."

END IF

// 3. Compute elements of L in column k ($i > k$)

FOR $i = k + 1$ TO n DO

$sum = 0$

 FOR $m = 1$ TO $k - 1$ DO

$sum = sum + L[i, m] * U[m, k]$

 END FOR

$L[i, k] = (A[i, k] - sum) / U[k, k]$

END FOR

END FOR

RETURN L, U

Assignment-1 Variant 1

Focus: Numerical Stability & Floating-Point Error Propagation

1. Assignment Description

In this variant, you will investigate how partial pivoting prevents catastrophic cancellation and how floating-point precision degrades when solving ill-conditioned linear systems.

2. Questions

- Audit and fix 'lu_starter_flawed.py' so it correctly implements $PA = LU$ decomposition with partial pivoting.
- Implement an error-tracking function that computes the Residual Infinity Norm: $\|Ax - b\|_{\infty} = \max (|Ax - b|_i)$

3. Test Cases & Matrices

Test Case 1.1: Small System Requiring Pivoting

Matrix $A = \begin{bmatrix} 0.0001 & 1.0 \\ 1.0 & 1.0 \end{bmatrix}$, Vector $b = [1.0001, 2.0]^T$

Exact Solution: $x = [1.0, 1.0]^T$

Test Case 1.2: The Hilbert Matrix Stress Test

Generate an $N \times N$ Hilbert matrix where $A_{i,j} = 1 / (i + j - 1)$. Generate b such that the exact solution is a vector of all ones. Do not use linear algebra packages.

4. Synthesis

Plot the residual norm $\|Ax - b\|_2$ as the size N of the Hilbert matrix increases from $N = 2$ to $N = 12$. At what dimension N does floating-point roundoff error begin to destroy accuracy, and why does this happen even with partial pivoting?

Assignment Variant 2

Focus: Computational Efficiency & Banded Matrix Optimization

1. Assignment Description

In many engineering applications (e.g., finite element analysis, heat transfer), linear systems are sparse and tridiagonal. In this variant, you will optimize standard $O(N^3)$ LU factorization to operate in $O(N)$ time for banded systems.

2. Questions

- Audit and fix 'lu_starter_flawed.py' to establish a working baseline solver.
- Create an optimized function 'lu_tridiagonal(A)' that skips elimination calculations for zero entries outside the main diagonal, upper subdiagonal, and lower subdiagonal.

3. Test Cases & Matrices

Test Case 2.1: 5x5 Tridiagonal System

Matrix A:

[4, -1, 0, 0, 0]

[-1, 4, -1, 0, 0]

[0, -1, 4, -1, 0]

[0, 0, -1, 4, -1]

[0, 0, 0, -1, 4]

Vector b = [15, 10, 10, 10, 15]^T

Test Case 2.2: Large 500x500 Sparse Tridiagonal System

Construct a 500x500 matrix with main diagonal = 2.0, upper subdiagonal = -1.0, lower subdiagonal = -1.0, and b = ones(500).

4. Synthesis

Measure execution time using Python's `time.perf_counter()` for $N = 50, 100, 200, 500$. Compare your optimized banded solver against standard LU decomposition. Construct a plot showing execution time vs N and verify that your algorithm achieves $O(N)$ complexity.

Assignment-3

Focus: Matrix Properties, Determinants & Explicit Inversion

1. Assignment Description

In this variant, you will extend LU decomposition to perform advanced matrix operations: evaluating the determinant $\det(A)$ without expensive recursive expansions and calculating the explicit inverse matrix A^{-1} .

2. Questions

- Audit and fix 'lu_starter_flawed.py' so it correctly computes $PA = LU$ decomposition.
- Implement 'compute_determinant(P, U)' using the property: $\det(A) = \det(P^{-1}) * \prod U_{i,i}$
- Implement 'compute_inverse(P, L, U)' by solving $AX = I$ column-by-column using forward and backward substitution.

3. Test Cases & Matrices

Test Case 3.1: 4x4 Symmetric Inversion Matrix

Matrix A:

[1, 2, 3, 4]

[2, 1, 2, 3]

[3, 2, 1, 2]

[4, 3, 2, 1]

Vector b = [10, 8, 8, 10]^T

Expected $\det(A) = -20.0$ | Exact Solution: $x = [1.0, 1.0, 1.0, 1.0]^T$

Test Case 3.2: Near-Singular Matrix Test

Matrix A = [[1, 2, 3], [4, 5, 6], [7, 8, 9.0000001]], Vector b = [6, 15, 24.0000003]^T

4. Synthesis

Explain why computing $\det(A)$ via the product of the main diagonal of U is $O(N^3)$ complexity, whereas standard cofactor expansion is $O(N!)$. How does your program handle singular or near-singular matrices during inversion?